CHARLOTTE

COLLEGE OF COMPUTING AND INFORMATICS

Group - 1 Project 2: CodePilot

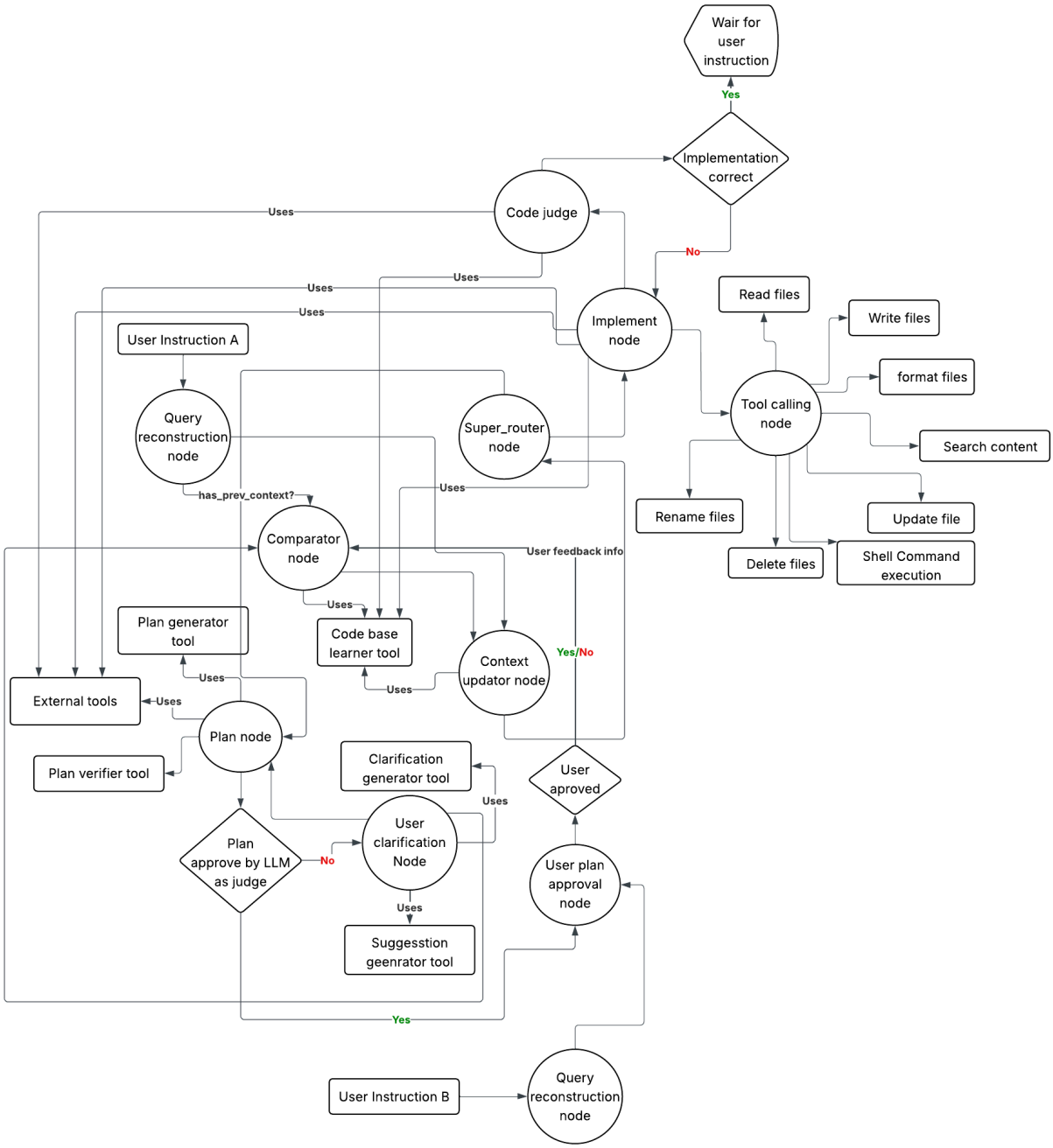
Gowshik Saravanan	gsarava2@charlotte.edu	801439710
Srushti Honnenahalli	shonnena@charlotte.edu	801057830
Devyansh Tailor	dtailor@charlotte.edu	801234175
Ilton da Costa Neto	idacosta@charlotte.edu	801382596
Hemanth Pasupula	hpasupul@charlotte.edu	801429998

Introduction

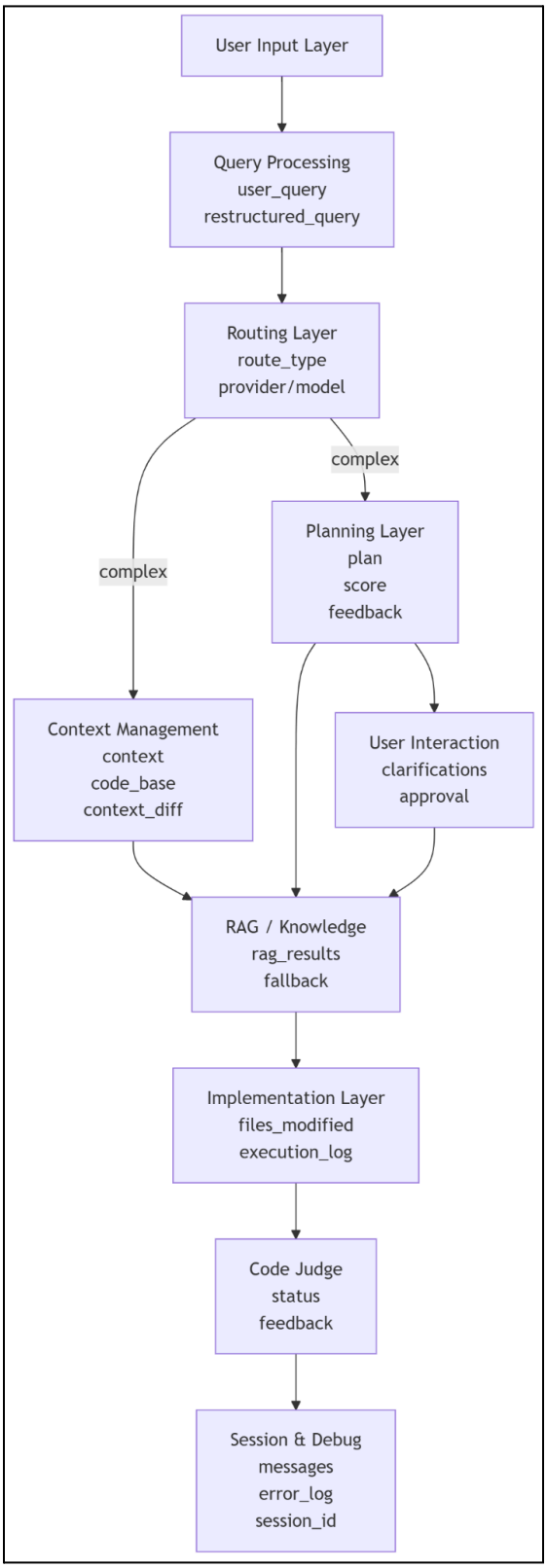
This document presents the system design for an AI-powered CLI coding agent built using LangGraph. The agent accepts natural language instructions from users and autonomously performs coding tasks such as reading, writing, searching, and modifying files within a codebase. The architecture follows a graph-based design where each step in the workflow is represented as a node, with conditional edges controlling the flow between them. The system is divided into two main paths — a fast path for simple tasks that go directly to implementation, and a detailed planning path for complex tasks that involves plan generation, LLM-based verification, and user approval before execution. Tools are capabilities that nodes call internally to perform actions like scanning the codebase, generating plans, or modifying files, keeping a clean separation between decision-making logic and action execution. A shared state object flows through the entire graph, allowing each node to read context from previous steps and write its results for downstream nodes. The design includes multiple feedback loops — an LLM judge catches low-quality plans before they reach the user, user approval ensures the agent never acts on misunderstood instructions, and a code judge verifies implementation correctness after execution. Safety mechanisms like iteration counters prevent infinite loops, and LangGraph's built-in checkpointing enables the agent to pause, resume, and maintain conversation history across sessions. This document covers the complete architecture including the system flow diagram, state schema, tool interfaces, prompt designs, and error handling strategies needed for implementation.

System Design

Architecture



State Schema Diagram



State Fields

Query Fields

Field Name	Data Type	Description	Default Value	Example Value
user_query	string	Raw input typed by the user	""	"Build a CLI coding agent"
restructured_query	string	LLM formatted version of user query, cleaned and made actionable	""	"The user is expecting to build a coding agent working in command line interface"

Context Fields

Field Name	Data Type	Description	Default Value	Example Value
context	dictionary	Dictionary holding current and previous context snapshots for comparison	{}	{ current: "Building agent", previous: "Coding agent for CLI to be built" }
code_base	dictionary	Complete codebase information including file tree, tech stack, configs, and dependencies	{}	{ file_tree: ["server.js", "routes/auth.js"], tech_stack: ["Node.js", "Express"], dependencies: ["express", "mongoose"] }
context_difference	string	Summary of what changed between previous and current codebase scans	""	"New file added: middleware/auth.js. package.json updated with json web token dependency"
has_prev_context	boolean	Whether the codebase has	False	True

		been scanned before in this session		

Routing Fields

Field Name	Data Type	Description	Default Value	Example Value
route_type	string	Task classification determined by super_router node	""	"complex"

Plan Fields

Field Name	**Data Type**	**Description**	**Default Value**	**Example Value**
current_plan	string	Step-by-step execution plan generated for complex tasks	""	"1. Install json web token 2. Create middleware/auth.js 3. Add login endpoint 4. Protect routes 5. Run tests"
plan_score	float	Quality score assigned by LLM judge (0.0 = bad, 1.0 = perfect)	0.0	0.91
plan_feedback	string	Feedback from LLM judge explaining what is wrong or could be improved in the plan	""	"Step 3 references the wrong file path. auth logic should be in middleware/ not routes/"
plan_history	list of strings	All previous versions of the plan for comparison across iterations	[]	["v1: Install JWT...", "v2: Install JWT + refresh tokens..."]
plan_iteration_count	integer	Number of times the plan has been regenerated or modified (safety counter)	0	2

User Interaction Fields

Field Name	Data Type	Description	Default Value	Example Value
clarification_questions	list of strings	Questions generated for the user when LLM judge rejects the plan to resolve ambiguity	[]	["Did you want JWT-based auth or OAuth?", "Should all routes be protected?", "Do you need role-based access?"]
suggestions	list of list of strings	Alternative approaches for each clarification question. suggestions[i] maps to clarification_questions[i]	[]	[["JWT tokens", "OAuth", "Session-based"], ["All routes", "Only user routes"], ["Yes with roles", "No"]]
user_approved	boolean	Whether the user approved the plan after reviewing it	False	True
user_feedback	string	Feedback or instructions from the user when they reject a plan or request changes	""	"No, I want OAuth not JWT. Also skip the refresh token part"
user_responses	list of strings	User's selected answers to clarification questions. user_responses[i] maps to clarification_questions[i]	[]	["JWT tokens", "Only user routes", "No, single role"]

Implementation Fields

Field Name	Data Type	Description	Default Value	Example Value
implementation_status	dictionary	Result from code judge containing status and feedback if incorrect	{}	{ status: "correct", feedback: "" } or { status: "incorrect", feedback: "Line 5 has wrong import" }
files_modified	list of strings	List of all files created, edited, or deleted during implementation	[]	["middleware/auth.js", "routes/auth.js", "server.js", "package.json"]
execution_log	list of dictionaries	Step-by-step log of what the implement node did, for code judge to review	[]	[{ tool: "read_file", input: "server.js", output: "success" }, { tool: "write_file", input: "auth.js", output: "created" }]
implement_iteration_count	integer	Number of times implementation has been retried after code judge rejection	0	1

MCP Integration Fields

Field Name	Data Type	Description	Default Value	Example Value
available_mcp_tools	list of dictionaries	All tools dynamically discovered from the 3 MCP servers at startup. Each entry contains tool name,	[]	[{ name: "read_file", server: "filesystem", description: "Read file contents" }, { name: "web_search", server: "tavily", description: "Search the web" }, { name: "query_python_docs", server: "custom-rag", description:

		description, schema, and source server		"Query Python documentation" }]
mcp_server_status	dictionary	Connection status of each MCP server. Checked at startup and monitored during execution	{}	{ filesystem: "connected", tavily: "connected", custom_rag: "connected" }

RAG Fields

Field Name	Data Type	Description	Default Value	Example Value
rag_query_results	list of dictionaries	Documentation chunks returned from the Custom RAG server for the current query. Contains matched text, source file, and relevance score	[]	[{ chunk: "asyncio.gather() runs multiple coroutines concurrently...", source: "asyncio.rst", score: 0.89 }, { chunk: "To handle exceptions in gather, use return_exceptions=True", source: "asyncio.rst", score: 0.85 }]
rag_fallback_used	boolean	Whether the Tavily web search was used as fallback because RAG returned	False	True

		insufficient or no relevant results		
rag_fallback_results	list of dictionaries	Web search results from Tavily when RAG fallback was triggered. Contains title, URL, and snippet	[]	[{ title: "Python asyncio documentation", url: "https://docs.python.org/3/library/asyncio.html", snippet: "asyncio is a library to write concurrent code..." }]

Provider Abstraction Fields

Field Name	Data Type	Description	Default Value	Example Value
active_provider	string	Currently active LLM provider. Agent dynamically switches between local and cloud based on task complexity	"ollama"	"groq"
active_model	string	Currently active model name within the provider	"codellama"	"llama3-70b"
provider_switch_reason	string	Why did the agent switch providers? Logged for debugging and transparency	""	"Task classified as complex, switching from local codellama to cloud groq/llama3-70b for better reasoning"

CLI Interface Fields

Field Name	Data Type	Description	Default Value	Example Value
execution_mode	string	Current execution mode. "confirmation" requires user approval for each tool call. "auto" executes all tool calls without intervention. User can toggle mid-session	"confirmation"	"auto"

Session and Debug Fields

Field Name	Data Type	Description	Default Value	Example Value
messages	list	Full conversation history including user messages, assistant responses, and tool call results	[]	[HumanMessage("Add auth"), AIMessage("I'll help..."), ToolMessage("file contents...")]
session_id	string	Unique identifier for the current session, used by LangGraph checkpointer for pause/resume	""	"session-abc-123"
error_log	list of dictionaries	Record of all errors encountered during the session for debugging	[]	[{ node: "implement", tool: "write_file", error: "Permission denied", timestamp: "2026-03-07T10:32:00Z" }]
current_node	string	Tracks which node is currently executing, useful for debugging and resuming after crash	""	"implement"

time_started	string	When the current task started, useful for timeout tracking	""	"2026-03-07T10:30:00Z"

Node Write Table

Node Name	Fields Written	When / Condition
query_reconstruction	user_query, restructured_query	Every new user message or follow-up instruction
query_reconstruction	time_started, session_id	Set once at the start of a new task
super_router	route_type, active_provider, active_model, provider_switch_reason	After classifying task. Switches to cloud model if complex, stays local if simple
comparator	context_difference, has_prev_context	When previous context exists and needs comparison
context_updator	code_base, context, has_prev_context	When scanning codebase fresh or updating after changes
plan_node	current_plan, plan_score, plan_feedback, plan_iteration_count, plan_history	After generating plan and scoring it. Queries RAG server for Python knowledge if needed
plan_node	rag_query_results, rag_fallback_used, rag_fallback_results	When plan_node queries RAG for Python docs. Writes fallback results if RAG insufficient
user_clarification	clarification_questions, suggestions, user_feedback	When LLM judge rejects plan and user needs to clarify
user_clarification	user_responses	After user selects answers to clarification questions
user_plan_approval	user_approved, user_feedback	When plan is shown to user and they approve or reject

implement	files_modified, implement_iteration_count, messages, execution_log	During code execution. Uses MCP filesystem tools instead of hardcoded tools
implement	rag_query_results, rag_fallback_used, rag_fallback_results	When implement node needs Python syntax/API info during coding
code_judge	implementation_status	After reviewing all changes made during implementation
(startup)	available_mcp_tools, mcp_server_status	At agent startup. MCP client connects to 3 servers and discovers all tools
(user command)	execution_mode	When user toggles between "confirmation" and "auto" mode mid-session
(all nodes)	current_node	Updated at the start of every node execution
(all nodes)	error_log	Appended whenever a tool call or LLM call fails

Node Read Table

Node Name	Fields Read	Why It Needs This
query_reconstruction	messages, user_feedback	Needs last user message to restructure; needs user_feedback if re-entering from rejection
super_router	restructured_query, active_provider	Needs a clean query to classify. Reads current provider to decide if switch is needed
comparator	context (previous), code_base	Needs previous snapshot to compare with current codebase state

context_updator	(none — scans fresh via MCP filesystem)	Calls codebase_learner tool via MCP filesystem server to get fresh data
plan_node	restructured_query, code_base, context, user_feedback, plan_feedback, user_responses, rag_query_results	Needs task + codebase + RAG results for Python knowledge. Uses feedback if regenerating
user_clarification	current_plan, plan_feedback, plan_score	Needs the plan and judge feedback to generate clarifying questions
user_plan_approval	current_plan, plan_score	Needs to show the plan and its quality score to the user
implement	current_plan, code_base, restructured_query, implementation_status, available_mcp_tools, execution_mode, rag_query_results	Needs plan to execute. Uses MCP tools from available_mcp_tools. Checks execution_mode for confirmation. Uses RAG for Python reference
code_judge	files_modified, current_plan, restructured_query, execution_log, rag_query_results	Needs what changed, the plan, action sequence, and RAG context to verify correctness

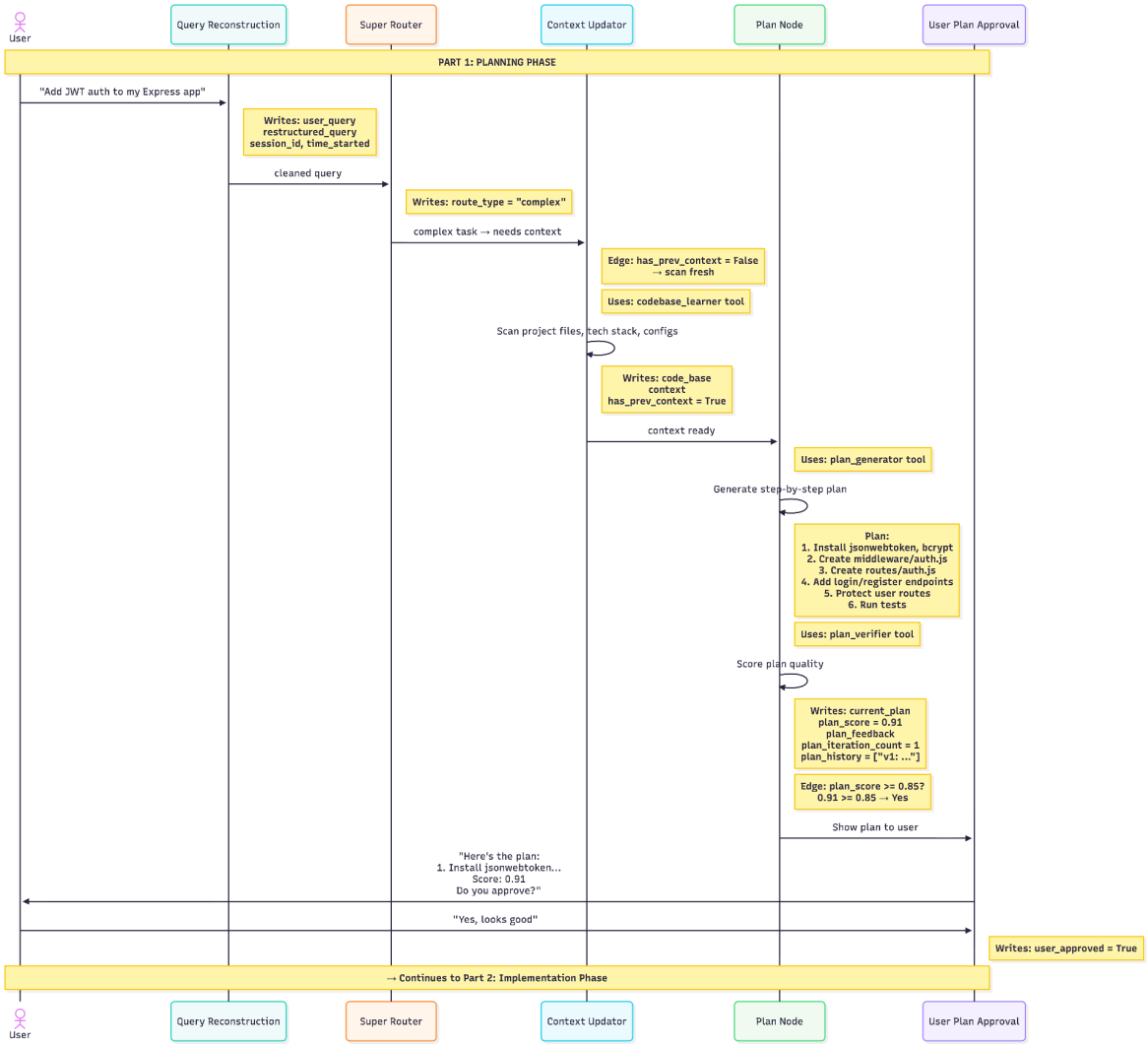
Conditional Edge Logic

Edge Name	**Field Checked**	**Condition for Yes**	**Yes Path**	**Condition for No**	**No Path**
has prev context?	has_prev_context	== True	Comparator node	== False	Context updator node
LLM judge approved?	plan_score	>= 0.85	User plan approval node	< 0.85	User clarification node

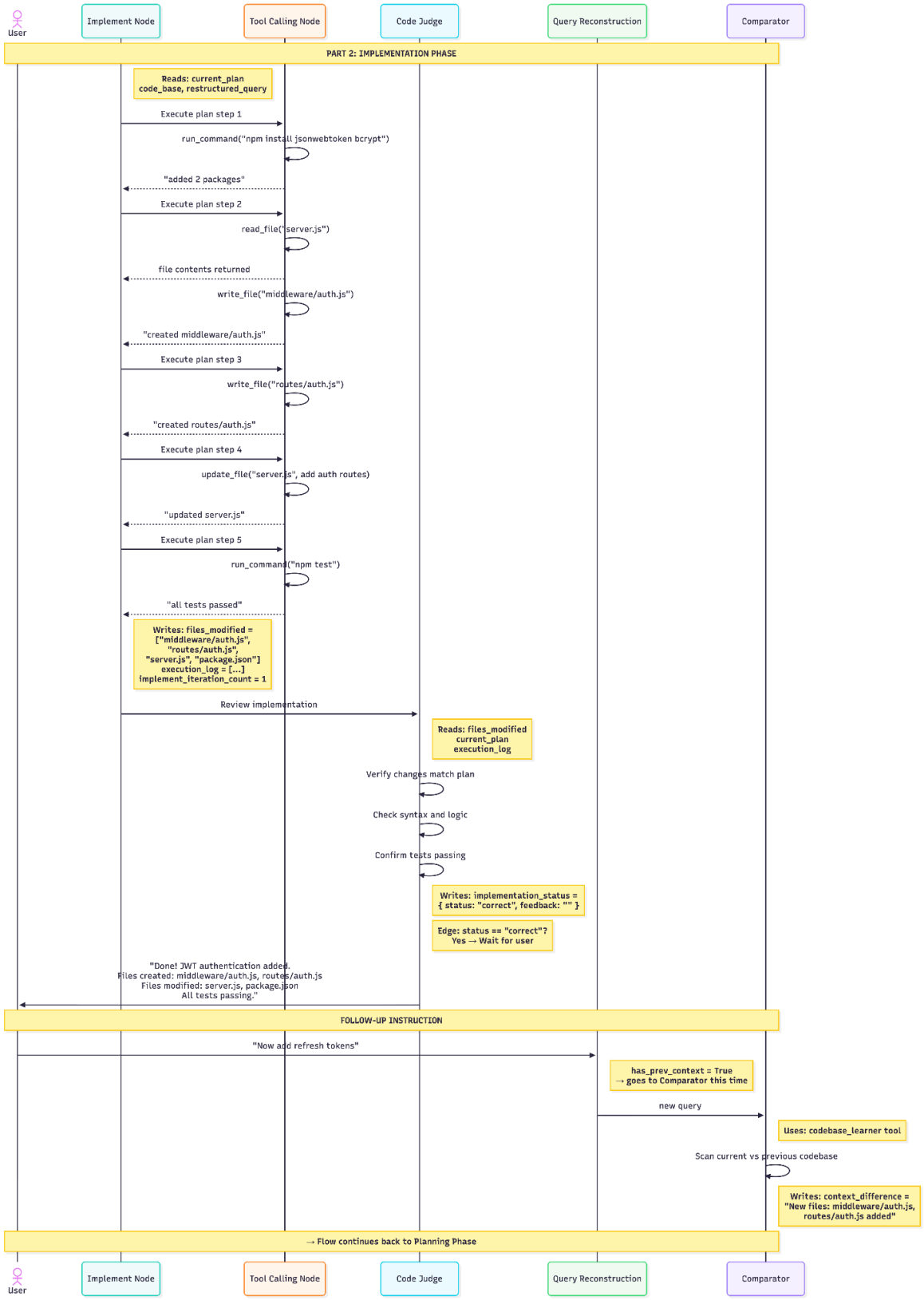
User approved?	user_approved	== True	Implement node	== False	User Instruction B → Query reconstruction
Implementation correct?	implementation_status.status	== "correct"	Wait for user instruction	== "incorrect"	Implement node (retry)
Route type	route_type	== "simple"	Implement node	== "complex"	has prev context? edge
Plan iteration safety	plan_iteration_count	< 4	Continue plan loop	>= 4	Force to user plan approval
Implement iteration safety	implement_iteration_count	< 5	Continue implement retry	>= 5	Wait for user with partial result
RAG sufficient?	rag_query_results	length > 0 and scores > 0.7	Use RAG results in prompt	empty or low scores	Fallback to Tavily web search
Provider switch needed?	route_type + active_provider	complex + local provider	Switch to cloud model	simple or already cloud	Keep current provider
Tool needs confirmation?	execution_mode	== "confirmation"	Ask user before each tool call	== "auto"	Execute tool immediately
MCP server healthy?	mcp_server_status	all servers "connected"	Proceed normally	any server "disconnected"	Log error, attempt reconnect, warn user

Sequence diagram

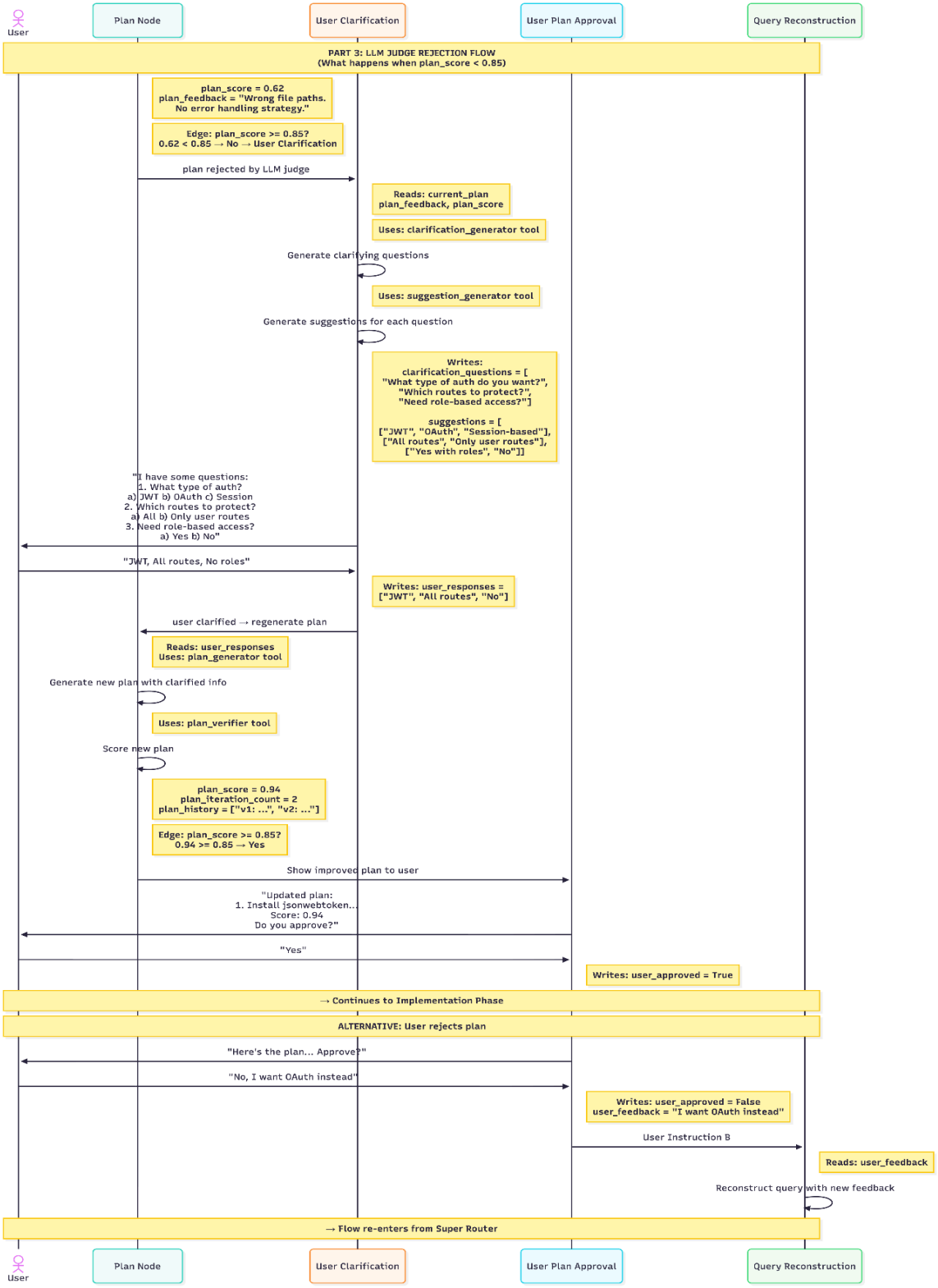
Planning Phase



Implementation Phase



LLM as a Judge Rejection flow

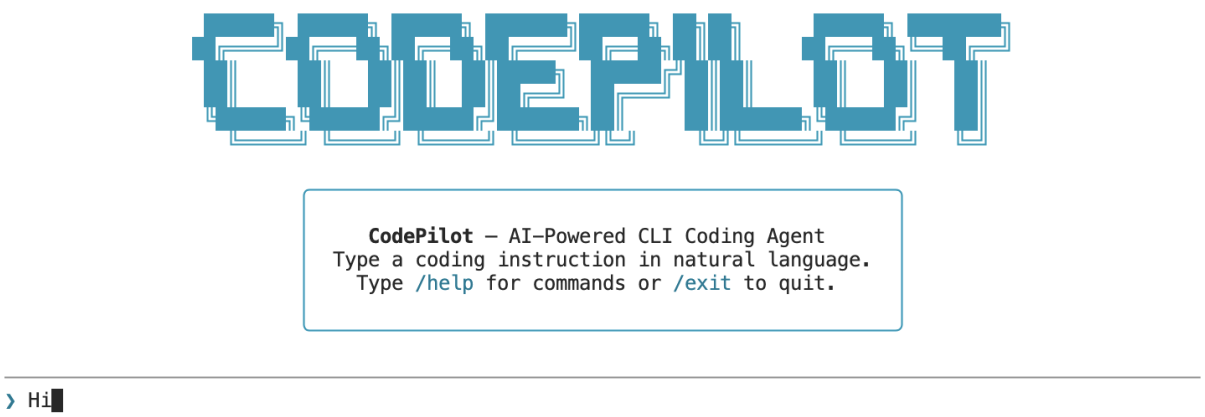


Provider Abstraction

The system uses LangChain's BaseChatModel interface to remain model-agnostic. All LLM calls go through a unified provider layer that supports local models via ChatOllama (e.g., codellama, llama3) and cloud providers via ChatGroq and ChatOpenAI. The active provider and model are configured through a config file. This abstraction allows swapping models without modifying the agentic loop, tool calling, or any other core logic.

CLI Interface

The system provides a terminal-based REPL interface with streaming responses, where LLM output is displayed token-by-token as it is generated. During tool execution, the CLI displays status indicators (spinners and labeled banners) showing which tool is being called and its result, making the agent's actions transparent to the user. The system supports two execution modes configurable in the config file: a confirmation mode that prompts the user to approve each tool call before execution, and an auto-execute mode that runs all tool calls without intervention. The interface uses color-coded output to distinguish between user input, agent responses, tool calls, and errors for readability.



MCP Integration

The system implements an MCP client that connects to three MCP servers at startup. The server configurations are defined in a config file. On initialization, the MCP client reads this config, establishes a connection to each server, and dynamically discovers all available tools. These tools are then unified into a single tool list provided to the LLM within the agentic loop, allowing the model to invoke any tool without knowing which server it originates from.
The three required MCP servers are:

- Filesystem Server (@modelcontextprotocol/server-filesystem): Provides tools for reading, writing, editing, and deleting files on the local machine. This is the primary way the agent interacts with the user's codebase.
- Tavily MCP Server: Provides a web search tool that allows the agent to search the internet for documentation, error solutions, and other relevant information during task execution.
- Custom RAG MCP Server: A locally-built MCP server that serves documentation from a chosen library's documentation through a vector database. This server handles document loading, chunking, embedding, and retrieval. The initial indexing process only runs once, and all future sessions query the pre-existing vector database. The server implements an advanced RAG technique to improve retrieval quality. Details of this server are described in the Custom RAG Server section.

Custom Advanced Rag

The system includes a locally-built MCP server that provides retrieval-augmented access to Python documentation. On the first run, the server executes a one-time indexing pipeline: Python documentation files are loaded, split into chunks, embedded using a sentence-transformers model, and stored in a ChromaDB vector database. All subsequent sessions skip indexing and query the existing database directly.

The server implements Fusion Retrieval as the advanced RAG technique. When a query is received, it runs two parallel searches: a keyword-based search using BM25 for exact term matching (e.g., asyncio.gather, dict.update) and a semantic vector search for conceptual queries (e.g., "how to handle exceptions in Python"). The results from both retrievers are merged and reranked using Reciprocal Rank Fusion (RRF) to produce a final set of relevant chunks, which are returned to the agent's MCP client and included in the LLM prompt.

The server exposes a single MCP tool (query_python_docs) that accepts a natural language query and returns the top relevant documentation chunks. During the planning

phase, the plan node queries this tool to gather Python-specific knowledge needed to generate accurate plans. During the implementation phase, the implement node queries it to get syntax details, module usage, and API references. If the RAG server returns insufficient or no relevant results, the agent falls back to the Tavily MCP server to search external resources for the needed information.

Insights

1. The "Human-in-the-Loop" Balance

The project makes a thoughtful decision about when to involve humans: clarification before planning, approval before execution. This is architecturally correct. Many agentic systems either ask too much (paralyzed by confirmation prompts) or too little (blindly executes). The threshold-gated approach, only ask if plan score < 0.85, is a good middle ground that keeps the agent autonomous without being reckless.

2. Intent vs. Implementation Gap

Query Reconstruction as the first node is architecturally smart. Users rarely articulate intent perfectly; restructuring before context-scanning means the agent anchors its understanding of the codebase to the intended task, not the raw query. This prevents downstream nodes from optimizing for the wrong goal.

3. RAG as the Current Bottleneck

The RAG corpus (data/docs/) appears to contain only sample.txt. For a coding agent, documentation quality directly limits plan quality. The retrieval infrastructure (BM25 + ChromaDB + RRF) is well-built, but the intelligence ceiling is set by what's in the documentation store. Scaling this with real Python stdlib docs, popular framework docs (Flask, FastAPI, SQLAlchemy), or project-specific documentation would dramatically improve plan relevance.

4. Stateless MCP Servers

Each MCP server spawns as a subprocess (npx, python). This is correct for isolation but means no state is shared between tool calls within a session. For implement nodes performing multi-file operations, this is acceptable. However, it also means no

transaction semantics: partial writes can't be rolled back if the implement loop fails mid-execution.

5. Latency Trade-offs in Provider Routing

Routing "simple" tasks to Ollama (local) and "complex" tasks to OpenAI/Groq is smart for cost but creates inconsistent UX: simple tasks respond in seconds, complex tasks take 5–30 seconds. The super_router classification quality directly determines user-experience variability. A misclassified "complex" task routed to Ollama will produce noticeably lower quality output.

Reflection

What Works Well

The architectural pattern (plan → verify → execute → judge) mirrors how experienced engineers approach tasks:

It's not just a wrapper around an LLM. It's a structured problem-solving loop. Each node has a single responsibility, and the conditional edges encode real decision logic (retry if plan is weak, re-implement if judge fails) rather than forcing a linear pipeline.

The multi-provider design shows practical thinking.

The system needs to work when OpenAI is rate-limited, when API keys aren't configured, or when running offline. Graceful degradation to Ollama means the system always produces something, even if quality varies, which is far better than a hard crash.

MCP adoption is forward-looking.

As the MCP ecosystem matures, swapping or adding tools requires no changes to core logic. The sandboxed filesystem access also makes the agent meaningfully safer than one with direct os module access.

Loop safety guards are explicit and correct.

Max 4 plan iterations, max 5 implement retries, and max 25 LLM steps per invocation prevent the runaway infinite loops that are a common failure mode in production agentic systems.

What Would Strengthen It

- Test coverage is the most glaring omission. Nodes, tools, and the RAG retriever all have deterministic-enough behavior to unit test. A test suite would catch regressions during iteration cycles, evidenced by the POML parser workaround and queue-based confirm mode, both of which emerged from discovered limitations mid-development.

- Observability is absent. Adding LangSmith tracing or structured logging to capture node transitions, plan scores, provider selections, and tool call outcomes would make debugging agentic failures far easier. Right now, a failed session requires reading through CLI output to reconstruct what happened.

- Idempotent implementation retries. When code_judge fails and triggers a retry, the implement node starts fresh without knowing what was already written. This can produce duplicate code, overwritten work, or compounding errors. A diff-aware retry prompt, which tells the implementing node what was already attempted and why it failed, would reduce wasted iterations.

- RAG corpus needs real content. The hybrid retrieval pipeline is well-architected, but sample.txt as the only documentation source limits it to a proof-of-concept. Indexing real documentation would shift RAG from a demo feature to a genuine quality improvement.

- Pickle-based checkpointing is fragile. pickle is sensitive to class definition changes, any refactor of AgentState silently breaks existing checkpoint files. A JSON or SQLite-backed store would be more resilient across code iterations.

Overall

CodePilot goes well beyond a "call the OpenAI API" course project. The combination of LangGraph state machines, MCP tool calling, hybrid RAG, and multi-provider fallback reflects genuine understanding of the challenges in production agentic system design — not just ML concepts. The design choices are mostly deliberate and well-reasoned; the limitations (POML fragility, pickle checkpointing, thin RAG corpus) are known trade-offs rather than oversights. The architecture is solid enough that addressing the gaps above would move it from a strong academic project toward something deployable.